Implementation Details

Pruning condition (c). A substantial ambiguity arises in pruning condition (c) for the lexicographic ejection enumeration. In [1], this condition is stated as

$$a_j^{new} = a_j \quad \text{and} \quad Q' \leq Q.$$

This condition is theoretically incomplete. For a correct pruning rule, checking $a_j^{new} = a_j$ together with $Q' \leq Q$ is insufficient. Let

$$I = \{\, i \mid a_i > l_i \,\}$$

denote the set of vertices in the original route at which the time-window constraint is already violated, and let

$$E_{k-1} = \{i^{(1)}, \ldots, i^{(k-1)}\}$$

denote the set of customers ejected up to the current step before the next increment. Then one must additionally require

$$E_{k-1} \cap I = \varnothing.$$

In other words, none of the already ejected customers may itself be a source of an original time-window violation. Without this additional condition, the pruning rule is incorrect, because it may discard branches of the enumeration that can still lead to an insertion–ejection combination optimal with respect to P_{sum}.

Inter-route and intra-route operations. The paper [1] states that the change in the time-window penalty for inter-route operators can be computed in $O(1)$ time for a single move. Since the number of inter-route candidates is of order $O(n^2)$, a complete enumeration of such modifications remains quadratic in the number of customers. In contrast, the paper pays considerably less attention to intra-route operations. This is precisely where the main implementation difficulty arises, because efficient penalty recomputation after an intra-route `Out-Relocate` or `EXCHANGE` is not an equally immediate consequence of the local nature of the move.

Naive asymptotics. Let the solution consist of routes $r_1, \ldots, r_m$, and let R_i denote the number of non-depot customers in route r_i. If, for every intra-route modification, the route penalty is recomputed from scratch after applying the move, then one check requires $O(R_i)$ operations. Since the number of possible intra-route relocations and exchanges in one route is of order $O(R_i^2)$, a complete enumeration of such candidates requires

$$O\left(\sum_{i=1}^{m} R_i^3\right),$$

which is equivalent to $O(n^3)$ in the worst case.

Intra-route `Out-Relocate`. For intra-route `Out-Relocate`, the implementation uses the following scheme. Let a vertex w in route r_i be fixed as the vertex to be moved, and let v denote the vertex before which w is to be inserted after being removed from its current position. Denote by $r_i \setminus \{w\}$ the route obtained by removing w from r_i. Then the exact penalty delta for the `Out-Relocate` operator decomposes as

$$\Delta_{\text{OutRelocate}}(v, w) = \Delta_{\text{Eject}}(w) + \Delta_{\text{Insert}}^{r_i \setminus \{w\}}(v, w),$$

where $\Delta_{\text{Eject}}(w)$ is the penalty change caused by removing w from the original route r_i, and $\Delta_{\text{Insert}}^{r_i \setminus \{w\}}(v, w)$ is the penalty change caused by inserting w before v into the route $r_i \setminus \{w\}$. In the implementation, the route $r_i \setminus \{w\}$ is constructed exactly once for a fixed vertex w, after which $\Delta_{\text{Eject}}(w)$ is also computed exactly once. Then, for all admissible positions v, only the term $\Delta_{\text{Insert}}^{r_i \setminus \{w\}}(v, w)$ is recomputed, and the cost of each such check is $O(1)$. Consequently, after one linear preprocessing step, the cost of evaluating all `Out-Relocate` candidates for a fixed w is $O(R_i)$, and a complete enumeration of all intra-route `Out-Relocate` moves in a route of length R_i requires $O(R_i^2)$ operations.

Intra-route `EXCHANGE`. For intra-route `EXCHANGE`, no equally strong improvement is available in the general case. Theoretically, if one insists on computing the exact delta for every pair of vertices (v, w) in the same route, then in the worst case a full penalty recomputation is still required on the route suffix starting from the first modified position. Therefore, exact processing of all intra-route exchanges in the general case preserves the asymptotic complexity

$$O\left(\sum_{i=1}^{m} R_i^3\right).$$

Nevertheless, in practice this issue is less critical on Gehring–Homberger instances, because the typical route length is relatively small there. As a result, a complete enumeration of all intra-route `EXCHANGE` moves requires even fewer elementary operations than a complete enumeration of all inter-route `EXCHANGE` moves, and therefore does not become a computational bottleneck.

Lower bound for intra-route `EXCHANGE`. In addition, in order to reduce the practical cost of intra-route `EXCHANGE`, the implementation uses a lower bound on the change in the time-window penalty. Let the vertices v and w in the same route be ordered so that v precedes w, and let v^-, v^+, w^-, w^+ denote their neighbors. First, only the local fragment

$$(v^-, w, v^+)$$

is recomputed, which yields a new earliest service time at vertex v^+, denoted by a'_{v^+}. If $a'_{v^+} \neq a_{v^+}$, then the remaining route suffix can no longer be reused without an additional full recomputation, and the computed partial value serves only as a lower bound on the full delta. If, on the contrary,

$$a'_{v^+} = a_{v^+},$$

then the contribution of the segment between v^+ and w^- remains unchanged and can be added exactly using the already available prefix values. After that, the local fragment

$$(w^-, v, w^+)$$

is recomputed in the same manner, which yields a new earliest service time a'_{w^+}. If, in addition,

$$a'_{w^+} = a_{w^+},$$

then the entire remaining route suffix coincides with the original one, and the constructed lower bound is equal to the exact delta. In this case, checking one intra-route `EXCHANGE` move costs $O(1)$ and does not require a full penalty recomputation on the route. If at least one of these equalities fails, the lower bound ceases to be exact and serves only as a preliminary pruning device.

Procedure `Perturb`. In the paper [1], the `Perturb` procedure is described only at a conceptual level: it is stated that I_{rand} random feasible modifications are applied to a feasible partial solution. However, the paper does not specify how such a feasible modification should be selected efficiently. This omission is important, because under direct random move generation a substantial fraction of attempts may turn out to be either inapplicable or infeasible.

Current implementation. In the current implementation, the `Perturb` procedure is organized as follows. Exactly I_{rand} times, two vertices v and w belonging to different routes are selected uniformly at random, and then a move type is selected at random from the set of inter-route operators. Applicability of the move and feasibility in terms of the penalty function are then checked; in effect, this means checking that the total penalty delta is zero. The modification is applied only if these checks succeed. Therefore, the actual number of performed changes is, generally speaking, smaller than I_{rand} and depends on the fraction of randomly generated candidates that happen to be feasible. In other words, in order to actually perform I_{rand} feasible modifications, this scheme generally requires generating a substantially larger number of random candidates.

Unimplemented scheme for selecting feasible modifications. A natural extension of this idea is to maintain the explicit set of feasible modifications. Let the solution consist of routes $r_1, \ldots, r_m$. For every ordered pair of routes (a, b), one may maintain an array

$$\text{mod}[a][b],$$

containing all feasible modifications of the form $\tau(v, w)$, where $v \in r_a$, $w \in r_b$, and τ is one of the considered move types. At the same time, one stores, for every pair (a, b), the size of the corresponding array,

$$M_{ab} = |\text{mod}[a][b]|.$$

Then the entire set of feasible modifications defines a discrete distribution in which each pair of routes is assigned weight M_{ab}. After prefix sums are built over the values M_{ab}, one feasible modification can be sampled in logarithmic or near-logarithmic time, depending on the chosen data structure used to maintain these sums.

After a modification has been applied, only one or two routes change, say r_v and r_w. Consequently, it suffices to remove obsolete modifications from all arrays

$$\text{mod}[\cdot][v], \quad \text{mod}[\cdot][w], \quad \text{mod}[v][\cdot], \quad \text{mod}[w][\cdot],$$

and then regenerate only those feasible modifications that involve the modified routes. If the lengths of these routes are R_v and R_w, then recomputing inter-route candidates with all remaining routes requires

$$O\big(n(R_v + R_w)\big),$$

whereas recomputing intra-route candidates for the routes r_v and r_w themselves requires, in the worst case,

$$O(R_v^3 + R_w^3).$$

Hence, one update of the set of feasible modifications after applying a move has total cost

$$O\big(n(R_v + R_w) + R_v^3 + R_w^3\big).$$

To support sampling according to the current values M_{ab}, it is sufficient to maintain a one-dimensional data structure for dynamic prefix sums on an array obtained by laying out the rows of the matrix (M_{ab}) consecutively one after another. Then sampling a pair of routes from the current discrete distribution, as well as updating the corresponding weights, can be performed in

$$O(\log(m^2)) = O(\log m),$$

where m is the current number of routes. This modification is not implemented in the present version, but it appears promising as a way to bring the actual number of performed perturbation steps closer to the nominal value I_{rand}.

Technical remarks.

Distance precomputation. From an asymptotic point of view, both computing a Euclidean distance from coordinates and reading a precomputed value from a table cost $O(1)$. However, these two primitives differ substantially in their constant factors. In the implementation, distances between all pairs of vertices are computed once when the instance is loaded and stored in a matrix. Thereafter, every access to c_{vw} reduces to reading a value from memory by vertex indices. In practice, this is faster than repeatedly computing the Euclidean distance, which involves floating-point arithmetic and a square-root operation. Since distances are used in nearly all local evaluations and in penalty recomputation, the benefit of precomputation propagates to all hot parts of the algorithm; this is also supported by flamegraph profiles, where distance accesses appear among the regularly invoked primitives on the hot path.

Arrays instead of lists. Initially, routes were represented as doubly linked lists, which allowed a structural route modification, excluding penalty recomputation, to be performed in $O(1)$. However, this advantage is immaterial in the present setting, because after each modification one still needs to recompute penalty characteristics at a cost of order $O(R_i)$. At the same time, a linked list is inferior to an array in terms of locality: it increases indirect pointer dereferences, degrades cache behavior and hardware prefetch efficiency, and makes sequential route traversal less efficient. A dense array representation, by contrast, accelerates precisely the part that dominates runtime. Profiling shows that the main cost lies in computing and updating penalty characteristics rather than in the structural route transformations themselves. Therefore, the gain from sequential data layout substantially outweighs the losses due to linear shifts of array elements upon insertion and deletion.

References

[1] Y. Nagata, O. Bräysy. A powerful route minimization heuristic for the vehicle routing problem with time windows. *Operations Research Letters*, 37(5):333–338, 2009.

[2] Y. Nagata, O. Bräysy, W. Dullaert. A penalty-based edge assembly memetic algorithm for the vehicle routing problem with time windows. *Computers & Operations Research*, 37(4):724–737, 2010.